

Segurança da API

DIM0547 — Sprint 3 · Vídeo 8 (Parte 2 de Autenticação)

Prof. Fernando · UFRN · 2026.1

Segundo vídeo de autenticação. No anterior implementamos **senhas com bcrypt** e o **login com JWT**. Aqui fechamos o que falta para a **Entrega Final: autorização, refresh tokens e proteção contra abuso** (através de limitação de taxa, ou *rate limiting*).

De onde paramos – e como usar este vídeo

Aula 07 (Vídeo 7) entregou a **identidade**:

- `POST /login` confere a senha com **bcrypt** e devolve um **JWT**
- `AuthMiddleware` valida o token e injeta o `userID` no contexto

O que falta para a **Entrega Final (29/06)**:

Tema	Exercício	Risco que resolve
Autorização por dono	ex03	OWASP API1 (BOLA)
Refresh tokens	ex04	sessão sem reduzir segurança
Limitação de taxa	ex05	OWASP API4 (abuso)

Está tudo em memória. Esta lista foca em **segurança**, não em persistência.

Mapa: a Lista 5+6 em cinco passos

Cada exercício acrescenta **uma** ideia de segurança. A ordem importa: cada um se apoia no anterior.

#	Exercício	Pergunta que responde
ex01	Senhas (bcrypt)	Como guardar senha sem guardar a senha? ✓ (Aula 07)
ex02	Access token (JWT)	Quem é o usuário em cada request? ✓ (Aula 07)
ex03	Autorização (ownership)	Este usuário pode acessar este objeto?
ex04	Refresh tokens	Como manter a sessão viva com segurança?
ex05	Limitação de taxa	Como impedir abuso de um endpoint?

Hoje resolvemos **ex03** → **ex04** → **ex05**, nessa ordem. **ex01** e **ex02** já estão prontos da aula passada — começamos com um recap rápido.

Parte 1 – Identidade: quem é o usuário

Recap: identidade já está resolvida

A Aula 07 fechou os dois primeiros exercícios:

- **ex01** — **bcrypt**: a senha nunca é guardada em texto puro; guardamos um *hash*. No login, `bcrypt.CompareHashAndPassword` confere se a senha bate com o *hash*.
- **ex02** — **JWT**: o login devolve um **access token** assinado (HS256), curto (15 min), enviado em cada request:

```
Authorization: Bearer eyJhbGc...
```

Três cuidados que já adotamos no **ex02**:

- **exp curto** — um vazamento tem impacto limitado no tempo
- **alg validado no parse** — defesa contra o ataque `alg:none`
- **erro genérico** no login (`invalid credentials`) — não revela se o e-mail existe

Esta é a base. O access token prova **quem** é o usuário. Mas provar quem é **não** é o mesmo que decidir o que ele pode fazer — é aí que entra a Parte 2.

Parte 2 – Autorização: o que o usuário pode

Autenticação ≠ Autorização

	Pergunta	Quem resolve
Autenticação	<i>Quem é você?</i>	login + JWT (Aula 07)
Autorização	<i>Você pode fazer isto?</i>	regra no handler

O JWT prova a identidade. Ele **não** decide o que essa identidade pode acessar — isso é responsabilidade de cada rota.

```
401 Unauthorized → não sei quem você é      (falta/inválido o token)
403 Forbidden    → sei quem é, mas não pode  (sem permissão)
404 Not Found    → o recurso não existe... ou você não pode saber que existe
```

Guarde o **404 com segundo sentido** — ele é a chave da defesa contra o BOLA, logo adiante.

OWASP API Security Top 10

A **OWASP** mantém listas dos riscos de segurança mais comuns. APIs têm a **sua própria** lista, separada da web tradicional: não há tela, o cliente fala direto com os endpoints, e os ataques são outros.

Hoje abordaremos as duas mais frequentes:

Risco	Nome	Onde, no projeto
API1:2023	<i>Broken Object Level Authorization (BOLA)</i>	acesso às notas (ex03)
API4:2023	<i>Unrestricted Resource Consumption</i>	abuso do login (ex05)

BOLA é, há anos, o **risco nº 1** em APIs — e o mais simples de explorar. É também o mais cobrado em revisão de código.

API1 – BOLA, explicado

Object Level Authorization = verificar, a cada acesso, se *aquele objeto específico* pertence a quem está pedindo.

O ataque (também chamado **IDOR**):

Ana está logada e acessa a própria nota:

GET /notes/7 (Bearer <token da Ana>) → OK, é dela

Ana troca o id na URL:

GET /notes/8 (Bearer <token da Ana>) → a nota 8 é de João

Se a API devolver a nota 8, ela **quebrou** a autorização por objeto. O token da Ana é válido (**autenticação** OK), mas ela **não é dona** da nota 8 (**autorização** falhou).

A falha não está no token — está em **esquecer de checar o dono**. Autenticar não é autorizar.

API1 – a defesa: 404, não 403

A regra de ouro: recurso de outro usuário → **404**, nunca **403**.

```
403 → "existe, mas você não pode"    ← confirma que a nota 8 existe
404 → "não encontrado"                ← a existência fica indistinguível
```

Como garantir na prática:

- O *store* busca pela chave **e** confere o dono na mesma operação
- "não existe" e "existe, mas é de outro" devolvem o **mesmo** erro
- O handler traduz esse erro único para **404**

SOLUÇÃO NO IDE – ex03 (ownership)

```
· internal/store/notes.go → GetForUser / UpdateForUser / DeleteForUser (checagem ok && ownerID ==
userID) · handler/notes.go → traduz ErrNoteNotFound em 404
```

Parte 3 – Sessão: como manter o login ativo

0 dilema do tempo de vida

exp curto (minutos)	exp longo (dias)
└ vazamento dura pouco	└ vazamento dura muito
└ usuário precisa autenticar toda hora (péssima UX)	└ usuário fica logado (boa UX, péssima segurança)

Segurança e boa experiência puxam para lados opostos. Um único token não resolve.

A saída é **separar responsabilidades**: um token para *acessar* (curto, usado o tempo todo) e outro para *renovar* (longo, usado raramente). Cada um otimiza um objetivo.

Dois tokens, dois papéis

	Access token	Refresh token
Função	Provar identidade em cada request	Obter um novo access token
Tempo de vida	Curto — 15 min	Longo — 7 dias
Usado	Em toda requisição	Só quando o access expira
É stateless?	Sim — JWT autocontido	Não — guardado (em <i>hash</i>) no servidor

O token que viaja muito (access) dura pouco. O que dura muito (refresh) quase não viaja — e, por ser guardado no servidor, **pode ser revogado**.

Por que o refresh é stateful

O access token é stateless: a assinatura basta para validá-lo. O refresh é o oposto — precisa de **estado** para podermos revogá-lo:

```
type RefreshToken struct {  
    Hash      string    // guardamos o HASH, nunca o token original  
    UserID    uuid.UUID  
    ExpiresAt time.Time  
    RevokedAt *time.Time // nil = ativo  
}
```

Um usuário tem **muitos** refresh tokens (um por dispositivo) — a relação 1:N que você já viu na Sprint 2, agora num *map* em memória. `RevokedAt == nil` é o estado "ativo".

0 fluxo: rotação e detecção de reuso

```
login          → access + refresh-1   (refresh-1 salvo como hash)
/refresh (rt-1) → revoga rt-1, emite access + refresh-2   (ROTAÇÃO)
/refresh (rt-1) → rt-1 já revogado → REUSO detectado → 401
                  + revoga TODOS os refresh do usuário
```

- **Rotação**: cada `/refresh` invalida o token usado e emite um novo. O cliente sempre tem o último.
- **Detecção de reuso**: se um refresh **já revogado** reaparece, é sinal de roubo → revoga toda a família e força novo login.

A ordem importa: a checagem de **reuso vem antes da de expiração**. Um token revogado que volta não é "expirado", é suspeito.

Logout: o que o refresh resolve

Token	Após o logout
Access token	Ainda válido — mas expira em minutos
Refresh token	Revogado imediatamente — não renova mais

O `/logout` revoga o refresh e devolve **204 sempre** — exista ou não o token (idempotente). Diferenciar vazaria se aquele token chegou a existir.

■ SOLUÇÃO NO IDE – ex04 (refresh tokens)

```
· internal/auth/token.go → GenerateRefreshToken (crypto/rand) / HashRefreshToken (SHA-256) ·  
internal/store/refresh.go → Insert / FindByHash / Revoke / RevokeAll · handler/auth.go → Login  
(emite o par), Refresh (rotação + reuso), Logout
```


Parte 4 – Proteção contra abuso

API4 – limitação de taxa

APIs gastam recursos a cada request: CPU, banco, e-mail, SMS. Sem limite, um cliente abusivo consegue:

- **Força bruta** de senha no `/login` — milhares de tentativas por segundo
- **Negação de serviço** (DoS) — afogar o servidor de requisições
- Estouro de custo em serviços pagos (envio de SMS, etc.)

A defesa é **limitar a taxa** por origem (IP):

```
até 5 req/s por IP, com folga (burst) de 10  
acima disso → 429 Too Many Requests (+ header Retry-After)
```

O `/login` é o alvo clássico de força bruta — por isso o rate limit entra exatamente nele. O algoritmo é o **token bucket**: cada IP tem um "balde" que reabastece a 5 fichas/s.

API4 – rate limiting no login

O middleware fica **na frente** do handler de login: lê o IP de origem, consulta o *limiter* daquele IP e, se estourou, corta a requisição com **429** antes de passar a chamada adiante.

- Um `rate.Limiter` por IP, guardados num `map`
- `429` com `Retry-After: 1` e corpo JSON `{"error":"rate limit exceeded"}`

SOLUÇÃO NO IDE – ex05 (limitação de taxa)

- `middleware/rate_limit.go` → `Middleware` (`SplitHostPort` → `getLimiter` → `Allow` → `429`) · `cmd/api/main.go`
→ onde o limiter é plugado só no `/login`

As correções de segurança, em um quadro

Correção	OWASP / risco	Onde
Autorização por objeto (404, não 403)	API1:2023 BOLA	ex03
Limitação de taxa no login	API4:2023 URC	ex05
Mensagem genérica "invalid credentials"	enumeração de usuário	ex02
<code>alg</code> validado no parse	ataque <code>alg:none</code>	ex02
Senha em bcrypt , refresh em SHA-256	vazamento de dados	ex01 / ex04
<code>JWT_SECRET</code> em variável de ambiente	segredo no repositório	config

Para a Entrega Final, **duas** correções já bastam.

Referências

Identidade e sessão

- `golang-jwt/jwt` — access tokens
- [OWASP — Session Management Cheat Sheet](#)
- `golang.org/x/time/rate` — token bucket

OWASP API Security

- [OWASP API Security Top 10 \(2023\)](#) — visão geral
- [API1:2023 — Broken Object Level Authorization](#)
- [API4:2023 — Unrestricted Resource Consumption](#)